PRE-FLIGHT: AI-POWERED CLOUD-READINESS COPILOT FOR GUIDEWIRE LEGACY ESTATES

21CSP302L- PROJECT

Submitted by

Rutba Sajad [RA2411003012262]
Saundarya Singh [RA2411003011364]

Under the Guidance of

Dr. Sandhia G K & Dr. Ranjani M

(Associate Professor, Department of Computing Technologies &
Assistant Professor, Department of Computing Technologies)

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTATIONAL INTELLIGENCE
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

OCTOBER 2026

Department of Computational Intelligence

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

SRM Institute of Science & Technology

Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with the student registration number and included with all assignments submitted. Work will not be marked unless this declaration is completed.

To be completed by the student for all assessments.

Degree / Course : B.Tech / 21CSP302L
Student Name : Saundarya Singh; Rutba Sajad
Registration Number : RA2411003011364; RA2411003012262
Title of Work : PreFlight: AI-Powered Cloud-Readiness Copilot for Guidewire Legacy Estates

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic Misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that we have met the following conditions:

- Clearly referenced / listed all sources as appropriate;
- Referenced and put in inverted commas all quoted text (from books, web, etc.);
- Given the sources of all pictures, data etc. that are not our own;
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present;
- Acknowledged in appropriate places any help received from others (e.g. fellow students, technicians, statisticians, external sources);
- Complied with any other plagiarism criteria specified in the Course handbook / University website.

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:
I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above. Saundarya Singh [RA2411003011364] _____ Rutba Sajad [RA2411003012262] _____

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY,
KATTANKULATHUR - 603 203

BONAFIDE CERTIFICATE

Certified that the 21CSP302L - Project report titled “PreFlight: AI-Powered Cloud-Readiness Copilot for Guidewire Legacy Estates” is the bonafide work of “Saundarya Singh [RA2411003011364] and Rutba Sajad [RA2411003012262]” who carried out the project work under the supervision of Dr. Sandhia G K and Dr. Ranjani M. Certified further, that to the best of our knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

<<Signature >> <<Signature >> <<Signature >>

SIGNATURE Dr. Sandhia G K Associate Professor DEPARTMENT OF COMPUTING TECHNOLOGIES	**SIGNATURE** Dr. Ranjani M Assistant Professor DEPARTMENT OF COMPUTING TECHNOLOGIES	**SIGNATURE** Dr. Niranjana G **PROFESSOR & HEAD** DEPARTMENT OF COMPUTING TECHNOLOGIES
EXAMINER 1 **Name & Signature**	**EXAMINER 2** **Name & Signature**	**EXAMINER 3** **Name & Signature**

ACKNOWLEDGEMENTS

We express our humble gratitude to Dr. C. Muthamizhchelvan, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and continued support.

We extend our sincere thanks to Dr. Leenus Jesu Martin M, Dean-CET, SRM Institute of Science and Technology, for invaluable support.

We wish to thank Dr. Revathi Venkataraman, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for support throughout the project work.

We encompass our sincere thanks to Dr. M. Pushpalatha, Professor and Associate Chairperson - CS, School of Computing, and Dr. C. Lakshmi, Professor and Associate Chairperson - AI, School of Computing, SRM Institute of Science and Technology, for their encouragement.

We are incredibly grateful to the Head of the Department, Department of Computational Intelligence, SRM Institute of Science and Technology, for suggestions and encouragement at all stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head and Panel Members, Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and suggestions.

We register our immeasurable thanks to our Faculty Advisor, Department of Computational Intelligence, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guides, Dr. Sandhia G K and Dr. Ranjani M, Department of Computational Intelligence, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under their guidance and for shaping the problem statement and setting a staged direction for the work. Their advice to establish the core migration workflow before adding optional agents has helped keep the proposal focused.

We sincerely thank all the staff members of the Department of Computational Intelligence, School of Computing, SRM Institute of Science and Technology, for their help during our project. We also acknowledge the authors and organizations whose public documentation on Guidewire APD, Cloud Retrofit, Cloud APIs, Cloud Assurance, Integration Gateway and Developer Assistants informed the literature review. Finally, we would like to thank our parents, family members and friends for their unconditional support.

Authors

Saundarya Singh; Rutba Sajad

ABSTRACT

InsuranceSuite estates accumulate years of product configuration, custom code, user-interface definitions and integrations. Migrating such an estate toward a cloud-oriented operating model first requires a clear understanding of what exists and how business features map to the technical artefacts that implement them. PreFlight is proposed as a local-first, review-oriented copilot that supports this discovery and migration-drafting work. It accepts a controlled set of project artefacts, extracts a traceable feature inventory, identifies dependencies, and prepares a draft target representation for a selected feature slice.

The architecture follows a staged three-agent design. Agent 1 performs migration and is divided into two cooperating subagents: Subagent A for application discovery, which maps the application's features and code relationships, and Subagent B for cloud reconstruction, which uses that map to prepare a reviewable target draft for a selected feature or line of business. Agent 2 reviews the available code and draft for violations and optimization opportunities. Agent 3 compares the original application with its cloud version. Because the latter two agents depend on the availability of original and target artefacts, the project begins with Agent 1.

The design is deliberately positioned alongside, and not as a replacement for, Guidewire's existing capabilities. Guidewire already provides APD conversion, Cloud Retrofit, Cloud Assurance, Cloud APIs and Guidewire-aware assistants. PreFlight is framed as a student-built prototype that demonstrates a staged, evidence-linked workflow on accessible sample artefacts, with clear traceability from source evidence to each proposed change.

Evaluation is defined on a controlled sample project, comparing discovered feature coverage, mapping accuracy, generated-draft validity, issue detection and reviewer effort against a documented manual baseline. Missing insurer-specific or third-party information will be reported as an evidence gap instead of being estimated.

Keywords: Guidewire, PolicyCenter, cloud migration, APD, LOB, multi-agent system, feature discovery, migration draft, cloud readiness.

TABLE OF CONTENTS

ABSTRACT v
TABLE OF CONTENTS vi
LIST OF FIGURES viii
LIST OF TABLES ix
ABBREVIATIONS x
1 INTRODUCTION 1
1.1 Introduction to the Project 1
1.2 Background and Platform Context 1
1.3 Problem Statement and Description 2
1.4 Motivation 2
1.5 Sustainable Development Goal of the Project 2
2 LITERATURE SURVEY 4
2.1 Overview of the Research Area 4
2.2 Existing Models and Frameworks 4
2.3 Limitations Identified from Literature Survey (Research Gaps) 6
2.4 Research Objectives 6
2.5 Product Backlog 7
2.6 Plan of Action (Project Road Map) 8
3 SPRINT PLANNING AND EXECUTION METHODOLOGY 9
3.1 Methodology Overview 9
3.2 Sprint I - Application Discovery 9
3.2.1 Objectives with User Stories of Sprint I 9
3.2.2 Functional Document 9
3.2.3 Architecture Document 10
3.2.4 Outcome of Objectives / Result Analysis 10
3.2.5 Sprint Retrospective 10
3.3 Sprint II - Cloud Reconstruction Draft 10
3.3.1 Objectives with User Stories of Sprint II 10
3.3.2 Functional Document 10
3.3.3 Architecture Document 11
3.3.4 Outcome of Objectives / Result Analysis 11
3.3.5 Sprint Retrospective 11
3.4 Later Agents and Extension Order 11
4 SYSTEM DESIGN AND ARCHITECTURE 13
4.1 System Context 13
4.2 Architecture Components 13
4.3 Multi-Agent Workflow 13
4.4 Core Data Model 14
4.5 Safety, Review and Failure Handling 14
4.6 Proposed Deployment Model 15
5 FUNCTIONAL DESIGN AND IMPLEMENTATION PLAN 16
5.1 Input and Output Contract 16
5.2 Discovery Subagent Design 16
5.3 Cloud Reconstruction Subagent Design 17
5.4 Agent 2 and Agent 3 Design 18
5.5 Interface and User Workflow 19
5.6 Implementation Stack and Modules 19
5.7 Planned Evaluation Measures 20
6 RESULTS AND DISCUSSIONS 22
6.1 Current Project Status 22
6.2 Expected Project Outputs 22
6.3 Illustrative Walkthrough 22

6.4 Discussion of Feasibility and Risks..... 23
6.5 Expected Contribution..... 23
7 CONCLUSION AND FUTURE ENHANCEMENT..... 24
REFERENCES..... 5
APPENDIX A CODING..... 27
APPENDIX B CONFERENCE PUBLICATION..... 28
APPENDIX C JOURNAL PUBLICATION..... 29
APPENDIX D PLAGIARISM REPORT..... 30

LIST OF FIGURES

CHAPTER TITLE PAGE NO.

1.1 Proposed project position within the broader modernization workflow. 3
3.1 Planned stages; no sprint is represented as completed. 12
4.1 Proposed staged architecture for Pre-Flight. 14
5.1 Planned artifact flow and review points. 16
5.2 Demonstration frontend — discovery inventory with evidence links (simulated sample data). 17
5.3 Demonstration frontend — cloud reconstruction draft with mapping table and patch preview (simulated). 18
5.4 Demonstration frontend — Agent 2 advisory review and Agent 3 comparison gates (simulated). 18
5.5 Demonstration frontend — review and approval with assumptions register (simulated). 19
6.1 Demonstration frontend — evaluation dashboard with coverage and omission checks (simulated). 21
2.1 Guidewire capabilities relevant to the project. 6
2.2 Evidence-based project gap and design response. 7
2.3 Research objectives and verification evidence. 7
2.4 Product backlog and acceptance conditions. 7
3.1 Sprint plan and agent sequence. 11
4.1 Agent and component responsibilities. 13
5.1 Inputs required for each level of claim. 16
5.2 Planned evaluation measures. 20
6.1 Expected outputs and current status. 22

ABBREVIATIONS

Abbreviation	Meaning
APD	Advanced Product Designer
API	Application Programming Interface
CAA	Cloud Assurance Assessment
CI/CD	Continuous Integration / Continuous Delivery
GWCP	Guidewire Cloud Platform
IG	Integration Gateway
JSON	JavaScript Object Notation
LOB	Line of Business
MCP	Model Context Protocol
PCF	Page Configuration Format
PS	Professional Services
SARIF	Static Analysis Results Interchange Format
SBT	Standards-Based Template
SI	System Integrator
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

InsuranceSuite applications typically embody several years of product configuration, custom Gosu code, user-interface configuration and integrations. In PolicyCenter, a single line of business may span product-model elements, entity definitions, user-interface artefacts, reference data and application programming interface resources. Any migration plan must therefore connect business features explicitly to the technical artefacts that implement them, rather than treating migration as a simple file translation.

PreFlight is proposed as a local-first, review-oriented copilot for this discovery and migration-drafting work. It accepts a controlled set of project artefacts, builds a feature inventory, identifies dependencies among artefacts, and prepares a draft target representation for a selected feature slice. The core migration capability is the first deliverable. To keep each stage independently reviewable, the architecture separates application discovery from cloud reconstruction so that an error in extraction can be distinguished from an error in transformation.

The initial prototype is not intended to replace Guidewire's product conversion services, Advanced Product Designer (APD), Cloud Retrofit or Cloud Assurance. Guidewire documents conversion of non-APD PolicyCenter product definitions into APD, Cloud API generation and supporting validation workflows [1-4]. PreFlight will instead demonstrate a student-built, staged process on accessible sample artefacts, with clear traceability from source evidence to each proposed change.

1.2 Background and Platform Context

Guidewire recommends APD for product design and provides a path for converting existing non-APD lines. Guidewire's December 2025 announcement describes a conversion service that uses tooling and expert services to reverse-engineer product definitions, import them into APD and generate Cloud APIs. The announcement gives an average conversion period of six to eight weeks for a monoline product of average complexity, including testing; this is Guidewire's stated estimate and is not presented as a benchmark for PreFlight [1]. Olos release information also identifies non-APD conversion and notes that Standards-Based Template (SBT) lines were not supported by that conversion capability at the time of that document [2].

Cloud Retrofit documentation covers non-APD-native products and describes template generation, product validation, code-generation configuration, endpoint generation and compile-error correction. The documented process can require human configuration and repeated generation when names or product structures do not align [3-6]. These existing capabilities establish the technical environment in which PreFlight is positioned. The prototype must therefore use precise language: it explores an assistant workflow around supplied artefacts and

1.3 Problem Statement and Description

Project teams need to inspect a legacy Guidewire application and understand how its product features, configuration and code relate before drafting a cloud-oriented equivalent. When the source estate is large or unfamiliar, the discovery work is difficult to organise. When only partial artefacts are available, a proposed migration may also omit behaviour that cannot be inferred from the files at hand. A useful copilot should therefore produce a traceable feature inventory and a reviewable migration draft, while explicitly identifying assumptions and unavailable evidence.

The problem addressed by this project is the design and prototyping of a staged multi-agent workflow that (a) explores an available Guidewire project to extract feature and dependency information, (b) maps a selected feature slice to a draft cloud target representation, and (c) leaves later verification agents as planned extensions that operate only when the corresponding source and target artefacts are available. The project will be measured on a controlled sample and will not claim production migration, business-rule equivalence or cloud deployment.

1.4 Motivation

The motivation arises from the supervisor's project direction and from the team's review of APD, Cloud Retrofit, Cloud APIs, Cloud Assurance and Qusar-era Developer Assistants. The proposed product should perform useful migration work rather than present only a list of warnings. Starting with application discovery and cloud reconstruction provides an end-to-end core workflow; additional checks can be introduced after that workflow produces a meaningful draft.

The initial project is deliberately limited to the supervisor-directed migration workflow. After the discovery and cloud-reconstruction stages are demonstrated, optional extensions may include APD mapping repair, an Integration Gateway scaffold, advisory compliance checks, schema reconciliation or controlled parallel simulation of candidate fixes. These extensions remain outside the first implementation milestone and will be reconsidered against available time, source material and Guidewire capabilities.

1.5 Sustainable Development Goal of the Project

The project is most directly related to Sustainable Development Goal 9: Industry, Innovation and Infrastructure. Its intended contribution is a research prototype for improving the engineering process used to modernise enterprise software. The project does not claim a measurable sustainability or productivity outcome at this stage. Any future impact claim would require evidence such as reduced review time, increased feature coverage or fewer unresolved migration defects in a defined evaluation.

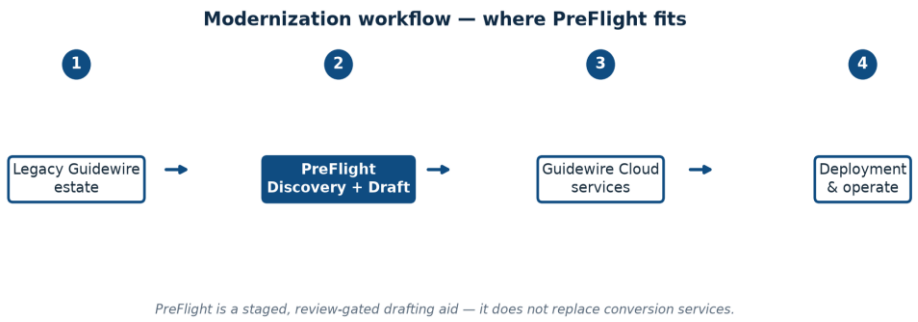


Fig 1.1: Proposed project position within the broader modernization workflow.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of the Research Area

The project sits at the intersection of enterprise application modernisation, product-model migration, static artefact analysis, code generation and AI-assisted development. These areas impose different evidence requirements. Product conversion reasons about the insurance product definition; code analysis reasons about configuration and source patterns; API generation exposes product resources; integration tooling connects the core system to external services. PreFlight's design keeps these tasks as distinct stages so that outputs from one stage can be checked before later stages rely on them.

The reviewed Guidewire sources show that migration is not a single file translation. The documented PolicyCenter process includes exporting or extracting a product representation, validating the visualised product, setting template or code-generation configuration, generating endpoints and correcting compile errors [3-6]. The documentation is therefore used to define the prototype's vocabulary and boundaries, rather than to imply access to internal tools or insurer environments.

2.2 Existing Models and Frameworks

2.2.1 APD CONVERSION AND CLOUD RETROFIT

Guidewire's APD conversion offering is the closest existing capability to the proposed core. It is designed to convert non-APD product definitions and preserve existing product logic while enabling APD and Cloud API workflows [1,2]. Public APD material describes a single product source and generated Cloud APIs for interoperability with Jutro and Integration Gateway [7]. This overlap means PreFlight cannot be presented as a new universal APD converter.

PolicyCenter Cloud Retrofit documentation describes both APD-assisted and non-APD processes. The APD path uses configuration files and product validation; the non-APD path may require product template extraction, validation, code-generation configuration, endpoint generation and compile-error correction [3-6]. For example, the current validation-error documentation explains that certain naming issues require changes to template or code-generation configuration followed by another generation step [5]. PreFlight's first prototype can use this workflow as a reference for migration stages and must record the evidence it used.

2.2.2 CLOUD APIS AND LOB RESOURCES

Guidewire Cloud APIs expose common system resources and can generate LOB-specific APIs from installed products. The public API reference explains how a configured environment can expose its actual API specifications, while the PolicyCenter documentation describes files generated for LOB endpoints [7,8]. This

matters for PreFlight because a cloud draft should be represented as structured artefacts - such as product metadata, API schema references and mappings - rather than a prose-only summary.

The generation process includes explicit configuration and validation steps. Documentation warns that generated files may need correction and regeneration when APD naming limits or product-model names create mismatches [5,6]. These documented examples are appropriate candidates for later prototype checks, but the initial migration agent should not hard-code every Guidewire rule before a core source-to-target workflow exists.

2.2.3 CLOUD ASSURANCE AND MIGRATION READINESS

Guidewire Cloud Assurance checks InsuranceSuite projects against Cloud Standards through Studio inspections and TeamCity Automated Cloud Assurance. The documented build can fail when violation thresholds are exceeded and generates an inspection report with details and code locations [9]. Agent 2 should therefore be an evidence-linked migration reviewer that explains relevant findings and optimisation opportunities in the context of the reconstruction draft; it must not claim to replace Cloud Assurance or report an official pass.

Guidewire's cloud migration materials describe technical upgrade, delivery and readiness work, and application testing, including integrations, database validation and performance [10,11]. The public migration discussion states that Guidewire is investing in automation and AI for upgrade errors and further integration and configuration changes [11]. The project's value proposition must therefore be framed as a research prototype for staged artefact discovery and draft reconstruction, not as a supposedly missing migration automation platform.

2.2.4 DEVELOPER ASSISTANTS AND AGENT WORKFLOWS

Qusar documentation describes Guidewire Developer Assistants as curated, version-aware knowledge interfaces available to MCP-compatible tools. It also documents an Agentic Framework for building tool-using agents and multistep workflows [12,13]. The August 2026 Developer Assistants article describes assistants for Gosu, Integrations and Jutro and states a direction toward MCP-connected capabilities that can interpret intent and propose coordinated changes [13]. These sources support modular assistant roles as a compatible design direction, without proving that PreFlight can connect to Guidewire's private MCP services.

The proposed system has three principal agents. Agent 1 performs migration and is split into two cooperating subagents: one explores the application and identifies features and code relationships; the other reconstructs selected functionality in a cloud-oriented draft. Agent 2 reviews available code for violations and optimisation opportunities. Agent 3 compares the original and cloud versions. Agents 2 and 3 depend on accessible source and target artefacts, so they follow the core migration workflow. Deterministic parsers and rules should handle structured checks where possible; a language model may explain findings but must not invent product behaviour.

2.2.5 INTEGRATIONS AND DOWNSTREAM CONSUMERS

Integration Gateway is documented as a cloud-managed service for integrating InsuranceSuite with internal and third-party systems [14]. App Events provide an outbound event approach, and Guidewire's Developer Assistant

for Integrations article describes building a route from an API file and producing mappings [13,15]. Jutro applications consume InsuranceSuite Cloud APIs through the Digital SDK, which is generated from API definitions [16,17]. These capabilities provide context for possible future integration and schema extensions.

The first PreFlight milestone will not attempt to generate a production Integration Gateway application or Jutro application. Such outputs require actual target versions, customer contracts, credentials, endpoint definitions and runtime validation. Later prototypes may generate non-deployable scaffolds or contract checks using synthetic specifications, clearly labelled as such.

2.3 Limitations Identified from Literature Survey (Research Gaps)

The literature review identifies a practical design constraint rather than a verified product gap. Public Guidewire documents cover APD conversion, Cloud Retrofit, validation, endpoint generation, Cloud Assurance, Cloud APIs, integrations and AI assistants. Public documentation does not establish whether Guidewire has an internal tool that orchestrates all migration steps for every insurer estate. The report therefore makes no claim that an end-to-end capability is absent.

Public documentation does not reveal every customer-specific migration process, insurer customisation or third-party contract. The prototype must therefore distinguish directly parsed facts from inferred relationships and unresolved questions. It will not assert that a vendor integration is compatible unless the relevant contract and target behaviour can be checked.

Table 2.1: Guidewire capabilities relevant to the project.

Evidence area	What public material establishes	What remains unknown for this project
APD conversion	Guidewire documents conversion of non-APD product definitions and Cloud API generation [1,2].	Private service implementation details, supported customer-specific transformations, and access for this student prototype.
Cloud Retrofit	Guidewire documents validation, configuration, endpoint generation and correction workflows [3-6].	Whether a student team has licensed environment access and which release can be used.
Cloud assurance	Guidewire documents CAA / TeamCity checks against Cloud Standards [9].	Exact internal rule corpus, thresholds and customer-specific extensions.
AI assistants	Qusar documents MCP-connected Developer Assistants and an Agentic Framework [12,13].	Authorisation to connect a student-built tool to Guidewire MCP services.
Insurer and vendor behaviour	Guidewire advises inventories for third-party integrations and data flows [18].	Customer-specific source code, business rules, vendor contracts and test credentials.

2.4 Research Objectives

The defensible academic question is whether a staged copilot can improve traceability between discovered legacy features and a reviewable cloud-oriented draft on a controlled sample. The tool will not attempt to prove universal feature equivalence. It will record source evidence, transformation rules, generated outputs, unresolved assumptions and reviewer decisions. This turns a broad migration promise into an evaluable prototype question.

Table 2.2: Evidence-based project gap and design response.

Observed condition	Project response	Evaluation evidence
Existing platform services already cover conversion and compliance areas.	Position PreFlight as an academic prototype for staged artefact discovery and draft reconstruction.	Explicit feature comparison and scope statement in report.
Migration outputs can depend on missing source or customer knowledge.	Record uncertainty and evidence gaps instead of guessing.	Percentage of findings linked to source files or marked unknown.
Later code comparison requires both source and target versions.	Keep comparison agent as a later phase and define a source/target availability gate.	Agent does not run without both baseline and candidate artefacts.
APD, CAA and AI assistants evolve by release.	Version-tag rule packs and sample fixtures.	Every finding reports the target version and rule version.

Table 2.3: Research objectives and verification evidence.

ID	Objective	Planned verification
O1	Design a source-artifact discovery workflow for the available Guidewire sample.	Feature inventory includes file, type, evidence location and confidence.
O2	Create a cloud-reconstruction subagent that produces a reviewable draft for one selected LOB feature.	Draft output is structured, version-tagged and linked to the discovered source feature.
O3	Define the evidence and inputs needed for the later violation/optimisation review and original-to-cloud comparison agents.	Each agent reports an input gap when source, target or version metadata is missing.
O4	Evaluate discovery coverage, reconstruction mapping, draft validity and reviewer effort on controlled fixtures.	Evaluation reports measured results on controlled fixtures.

2.5 Product Backlog (Key User Stories with Desired Outcomes)

Table 2.4: Product backlog and acceptance conditions.

Priority	User story	Acceptance condition
P0	As a developer, I can select an available sample Guidewire project and run discovery.	Unsupported files are reported; input files remain unchanged.
P0	As a reviewer, I can see detected	Each inventory item contains source

	features and the files supporting each one.	path and extraction method.
P0	As a developer, I can request a draft cloud representation for one selected feature.	Draft is generated from explicit templates/rules and linked to source evidence.
P0	As a reviewer, I can approve, reject or edit generated changes.	No target patch is treated as accepted without human review.
P1	As a migration lead, I can view unresolved information gaps.	Unknown business or vendor behaviour is marked as unverified.
P2	As a migration lead, I can run the violation/optimisation reviewer and compare original code with the cloud draft.	Both agents report unavailable inputs and cite the artefacts used.
P3	As a team, we can add optional simulation, APD, integration and schema capabilities.	Extensions are evaluated only after the three-agent workflow is demonstrated.

2.6 Plan of Action (Project Road Map)

The project is planned as an incremental build. Sprint I explores the available Guidewire application and produces a feature and code-reference inventory. Sprint II uses that inventory to reconstruct one selected feature in a cloud-oriented draft. The next phase adds Agent 2 for violation and optimisation review and Agent 3 for comparison between the original and cloud versions, provided both sets of artefacts are available. Evaluation follows on controlled fixtures. Optional extensions remain outside the core sequence until it works end to end.

For project planning, Scrum terminology is used to organise work into time-boxed increments; it does not imply that the team has completed any sprint. The proposed roadmap is a plan, not a record of past execution. The Scrum Guide is cited as the methodological reference for this organisation [20].

CHAPTER 3

SPRINT PLANNING AND EXECUTION METHODOLOGY

3.1 Methodology Overview

The report follows an iterative prototype methodology. Each increment begins with a small, testable user outcome, produces a reviewable artefact, and ends with a retrospective on the workflow. The team will use fixed sample inputs and expected outputs so that changes to parsers, mapping rules or prompts can be compared consistently. The first increment focuses on discovering the application; the second produces the first cloud reconstruction draft.

The system is intentionally organised around input/output contracts. The discovery subagent receives a project root and version notes and emits an inventory. The reconstruction subagent receives a selected inventory item plus target conventions and emits a draft patch and rationale. This separation permits the team to distinguish an extraction error from a transformation error.

3.2 Sprint I - Application Discovery

3.2.1 OBJECTIVES WITH USER STORIES OF SPRINT I

Sprint I will implement the first subagent described by the supervisor: exploring the breadth and functionality of the Guidewire application available to the team. The subagent will locate supported configuration and product artefacts, build a file index, identify candidate LOB features, and collect references to code or integrations that appear related. Its aim is to create an understandable map of the available application, not to claim complete semantic comprehension of every Gosu class or insurer rule.

As a developer, I can select a project directory or archive and receive an inventory of recognised artefacts.

As a reviewer, I can trace a feature to the source file, entity, configuration entry or documentation fragment that supports it.

As a migration lead, I can see unsupported files, ambiguous links and missing inputs before reconstruction starts.

3.2.2 FUNCTIONAL DOCUMENT

The discovery stage will use deterministic file enumeration and format-specific readers where possible. Extensible Markup Language (XML), JavaScript Object Notation (JSON), YAML and source files will be treated according to explicit parsing rules. A language model may assist with summarising a selected code unit, but every summary must include file references and must be labelled as a hypothesis if static evidence does not prove behaviour. No network access to an insurer tenant is required for the offline sample mode.

Expected discovery outputs include a manifest of files, a feature inventory, a dependency list, a list of extraction warnings, and a provenance map. The manifest records hashes and project version metadata so later stages can identify the exact input snapshot. The system will not modify the input project during discovery.

3.2.3 ARCHITECTURE DOCUMENT

The discovery subagent is designed as a bounded worker called by a local orchestrator. It will return machine-readable JSON plus a human-readable summary. Each finding contains an identifier, category, source location, observed text or parsed key, confidence and unresolved questions. The feature inventory will distinguish product-model facts from inferred behaviour.

3.2.4 OUTCOME OF OBJECTIVES / RESULT ANALYSIS

The outcome is assessed on a fixed sample containing known product artefacts, code references and a small number of deliberate ambiguities. Planned measures are extraction coverage, source-link accuracy, unsupported-file handling and reviewer agreement on feature groupings.

3.2.5 SPRINT RETROSPECTIVE

The retrospective will be completed after Sprint I. The team will record which file formats were understood, which feature relationships were difficult to establish, and which unsupported inputs should be declared in the interface. Any scope reduction will be made explicit before Sprint II begins.

3.3 Sprint II - Cloud Reconstruction Draft

3.3.1 OBJECTIVES WITH USER STORIES OF SPRINT II

Sprint II implements the second subagent from the supervisor's sequence: replicating selected functionality in a cloud-oriented representation. The agent consumes the discovery inventory and prepares a draft for one selected LOB feature. To keep the prototype measurable, the first version will target a deliberately bounded feature slice such as a Personal Auto coverage or vehicle-related element, depending on the available sample artefacts.

As a developer, I can select one discovered feature and ask for a cloud reconstruction draft.

As a reviewer, I can inspect the source-to-target mapping and see which transformation rule produced each proposed change.

As a reviewer, I can reject ambiguous transformations and leave them unresolved instead of accepting a guessed business rule.

3.3.2 FUNCTIONAL DOCUMENT

The reconstruction subagent will read the selected feature record, a versioned rule pack and a target template. It will produce a draft artefact, a proposed change set, a mapping table and a list of assumptions. Deterministic transformations will handle known structural conversions. A model may explain or classify an item, but an output

cannot be labelled verified unless a check can compare it with explicit source evidence or a runnable target validator.

The minimum viable product will not deploy to Guidewire Cloud or invoke APD services. If the team obtains an authorised environment, a future validation adapter may run the target checks. Without such access, all generated outputs will be labelled local drafts or simulations. This boundary prevents a mock output from being misrepresented as a successful PolicyCenter generation or live API response.

3.3.3 ARCHITECTURE DOCUMENT

The reconstruction subagent receives the selected feature and relevant evidence rather than attempting to transform the entire application at once. It emits a patch proposal with source anchors and target file paths. A review layer shows the difference and requires approval before writing generated files. The output includes a machine-readable mapping manifest so Agent 3 can later compare the cloud draft with the original, when both are available.

3.3.4 OUTCOME OF OBJECTIVES / RESULT ANALYSIS

Evaluation measures whether the draft is structurally valid, whether all selected source fields have a target disposition, and whether the reviewer can identify unsupported assumptions. Success is not defined as matching Guidewire's private conversion service; it is defined as correctly processing the chosen fixture and explaining limitations.

3.3.5 SPRINT RETROSPECTIVE

The Sprint II retrospective will examine the quality of the generated difference, the number of manual corrections, and the causes of any lost or incorrectly mapped fields. The team will decide whether the next increment should strengthen feature reconstruction or add a cloud-readiness checker. It will not add further ideas solely to increase feature count.

3.4 Later Agents and Extension Order

Agent 2 reviews the migrated draft for violations and optimisation opportunities using available rules and source locations. Agent 3 compares original and cloud artefacts to identify missing, changed or unmatched functionality. Both agents depend on the original source and a target draft with version context. When either artefact is unavailable, the report must state that the check could not be completed rather than imply that no issue exists.

Table 3.1: Sprint plan and agent sequence.

Stage	Agent or work package	Dependency	Status in this report
1A	Application discovery subagent	Available Guidewire project artefacts	Core design milestone
1B	Cloud reconstruction	Discovery inventory and	Core migration draft

	subagent	target conventions	
2	Violation and optimisation reviewer	Original and cloud artefacts plus applicable rules	Later stage; gated on inputs
3	Original-to-cloud comparison	Original source and reconstructed cloud version	Later stage; gated on inputs
Future	Simulation, APD, integration and schema extensions	Core workflow accepted and suitable contracts or fixtures available	Optional work

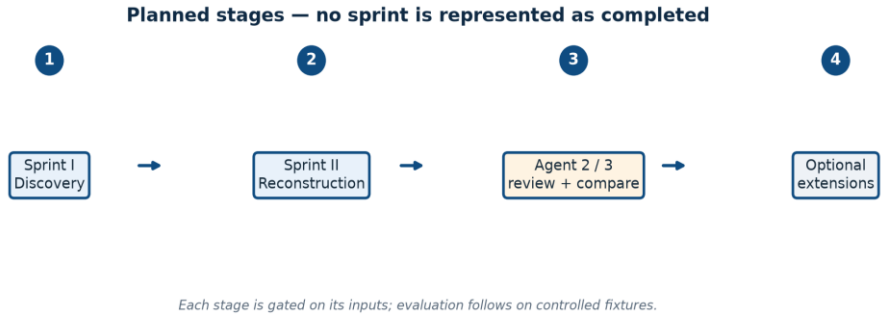


Fig 3.1: Planned stages; no sprint is represented as completed.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 System Context

PreFlight is designed as a local project-analysis application with a user interface, an orchestrator, specialist agents, an artefact store and a review surface. The operator selects a source snapshot and target release context. The application then invokes the discovery subagent, presents the resulting feature inventory, and allows the operator to request a reconstruction draft for a selected item. All generated changes are staged for review.

A local-first prototype is appropriate because insurer code, configuration and vendor specifications can be confidential. The design assumes that project artefacts remain within the user-selected workspace. If a language model is used, the implementation must make the data route explicit and must not transmit private source code without authorisation. The initial academic version can use local parsing and a mock language-model adapter.

4.2 Architecture Components

Table 4.1: Agent and component responsibilities.

Component	Responsibility	Output
Workspace intake	Select project snapshot, record versions, reject unsupported archives safely.	Input manifest and hashes.
Orchestrator	Run bounded workers in order and pass only required evidence.	Run state and provenance chain.
Discovery subagent	Extract features, code references, product structures and dependencies.	Feature inventory and evidence map.
Reconstruction subagent	Apply versioned mapping rules and target templates to selected feature.	Draft patch, mapping report, assumptions.
Review and approval	Display proposed changes, support accept/reject/edit, record reviewer decision.	Approved or rejected change set.
Agent 2 and Agent 3	Review violations/optimisations and compare original with cloud artefacts when available.	Evidence-linked findings and explicit input gaps.

4.3 Multi-Agent Workflow

The core migration agent is split into two subagents as advised by the supervisor. Subagent A explores the available application and constructs a source-grounded feature map. Subagent B uses a selected feature record to prepare a target-side draft. The orchestrator controls the order and stores each intermediate artefact. This decomposition makes it possible to inspect whether a poor output originated in feature discovery or in

reconstruction.

Agents 2 and 3 follow the core pipeline. Agent 2 checks original and cloud code for selected violations and optimisation opportunities using a cited rule set. Agent 3 compares the original and target artefacts and reports preserved, missing, changed or unresolved functionality. The comparison agent must not run if either side is absent. Where a vendor contract is missing, the system reports unknown compatibility rather than assuming that an integration migrated correctly.

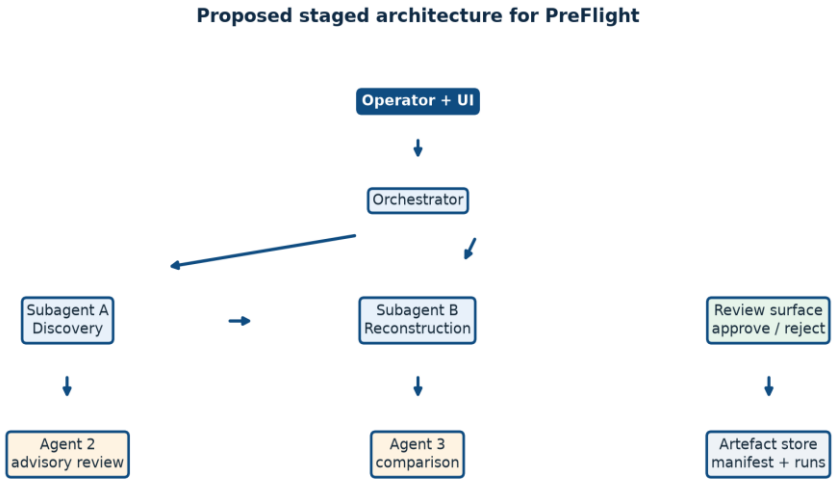


Fig 4.1: Proposed staged architecture for PreFlight.

4.4 Core Data Model

A feature record will include a stable internal identifier, feature name, line of business, source artefact paths, observed product elements, related code references, external dependency references and evidence status. Each extracted property will record its source location and extraction method. The record will also hold a confidence label: directly parsed, linked by explicit configuration, or inferred for review.

A mapping record will connect a source property to a target property or to an explicit disposition such as unsupported, intentionally omitted, or needs business decision. Every mapping will include the transformation rule identifier, source and target release context, and reviewer status. This makes the result inspectable and supports later source-target comparison without relying on a generated narrative alone.

4.5 Safety, Review and Failure Handling

PreFlight must not silently modify source artefacts. It will read the selected snapshot, write proposals to a separate output directory, and show a difference view. The operator must review changes before accepting them. A failed parser should create a visible extraction warning rather than an empty success report. An unknown field must remain unresolved unless a reliable rule or human decision supplies its target meaning.

The prototype will use a controlled sample and synthetic dependencies for repeatable demonstration. It will not store credentials, call a real carrier's third-party services, or deploy to a live Guidewire tenant. Future access to an authorised PolicyCenter Cloud environment would be required to validate true endpoint generation or runtime behaviour.

4.6 Proposed Deployment Model

For the academic demonstration, PreFlight can run as a local web application with a backend worker process and a versioned rule library. The file-upload surface is intended for selected project archives only. A run record stores input hashes, rule versions, extracted inventory, proposed patch and reviewer actions. A future enterprise build could integrate with an approved repository workflow or MCP tool, but that connection is outside the current scope.

CHAPTER 5

FUNCTIONAL DESIGN AND IMPLEMENTATION PLAN

5.1 Input and Output Contract

The prototype must make its input expectations visible. A valid run requires an accessible Guidewire project snapshot or a controlled sample, the PolicyCenter or InsuranceSuite release where known, and an explicit target context for the migration draft. For source-target comparison, both snapshots are required. For external integration review, API specifications or equivalent partner documentation are required for compatibility claims.

Table 5.1: Inputs required for each level of claim.

Artifact	Required for core discovery	Required for reconstruction or verification
Guidewire project files	Yes	Yes, for selected feature
Release / version metadata	Preferred	Yes for target-aware rules
APD template or product model	If present, parse and catalogue	Required for APD-specific mappings
Original insurer custom code	If supplied, include as source evidence	Required for insurer-specific equivalence claims
Third-party API schema / contract	No for basic dependency discovery	Required for API compatibility verification
Target cloud code or tenant	No for a local draft	Required for real compile/runtime validation

5.2 Discovery Subagent Design

The discovery subagent will begin with a file inventory and type detection. It will parse supported structured files and scan source files for known Guidewire constructs and references. The implementation should preserve exact file paths and line ranges where practical. A feature grouping is proposed from explicit identifiers and relationships; when the relationship is inferred from naming alone, the record must carry a lower confidence and require human review.

The discovery summary should answer: which line-of-business or product elements were found, which source files implement or configure them, which dependencies were found, and which file categories could not be interpreted. The goal is not to summarise every line of code. It is to make the migration scope inspectable enough that a developer can choose a feature slice for the next stage.

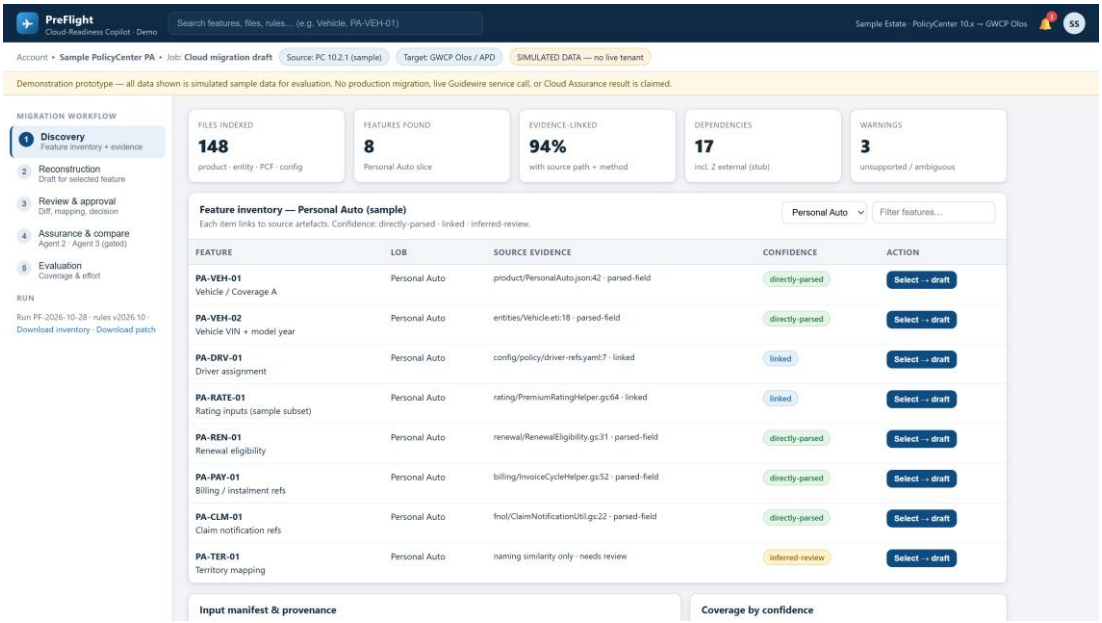


Fig 5.2: Demonstration frontend — discovery inventory with evidence links (simulated sample data).

Fig 5.2 shows the running demonstration frontend in the discovery view. The feature inventory lists eight Personal Auto items with source evidence paths and confidence labels (directly-parsed, linked, inferred-review), together with the input manifest and coverage chart. All values are simulated sample data; the input project is never modified during discovery.

5.3 Cloud Reconstruction Subagent Design

The reconstruction subagent will use explicit mapping rules and templates. Each rule must name its target Guidewire release or target artefact convention. A mapping rule can preserve a field, rename it with an explicit code-generation mapping, flag it for business review, or mark it unsupported for the current prototype. The output will include proposed files and a patch rather than directly overwriting existing project configuration.

For an initial Personal Auto demonstration, one feature can be selected from the available sample. The agent might map a vehicle or coverage field from a source representation into a target product or API representation. Because public Guidewire documentation describes APD and Cloud API behaviours, the prototype can use those public patterns as references [3-8]. It must not claim the output is deployable unless it passes a real target environment's validation and build.

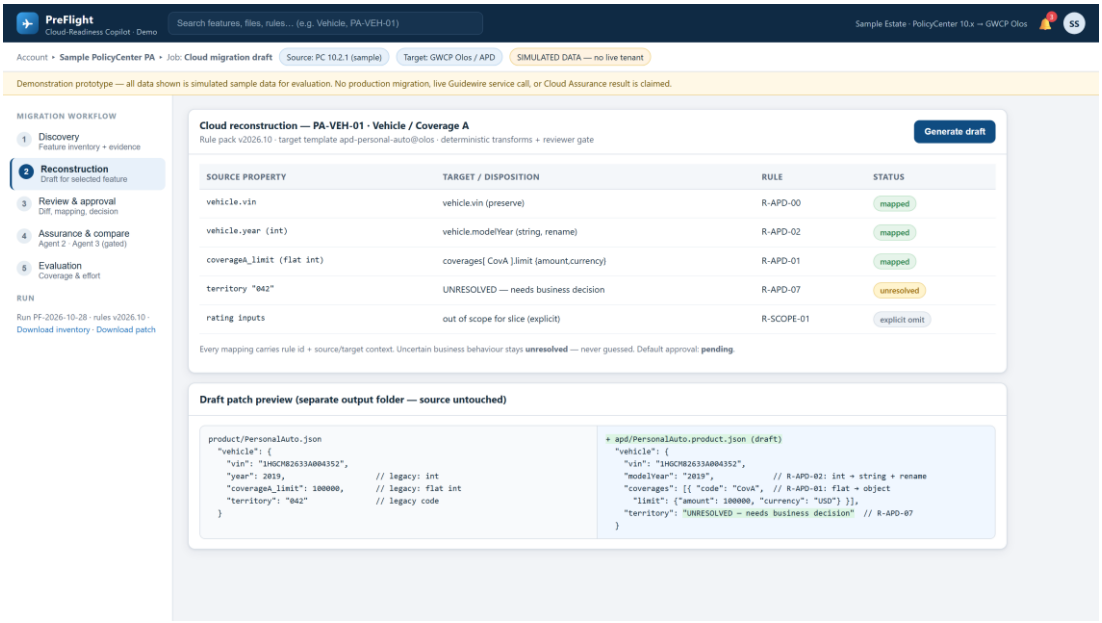


Fig 5.3: Demonstration frontend — cloud reconstruction draft with mapping table and patch preview (simulated).

Fig 5.3 shows the reconstruction view for feature PA-VEH-01. The mapping table links each source property to its target disposition with rule identifiers (R-APD-01, R-APD-02, R-APD-07), and the patch preview contrasts the legacy representation with the draft target. Uncertain behaviour remains unresolved pending reviewer decision.

5.4 Agent 2 and Agent 3 Design

Agent 2 will examine the available original and reconstructed artefacts for selected violations and optimisation opportunities. Every finding should identify the rule source, target version, code location, severity and rationale. Its report is advisory and must remain distinct from an official Cloud Assurance result [9].

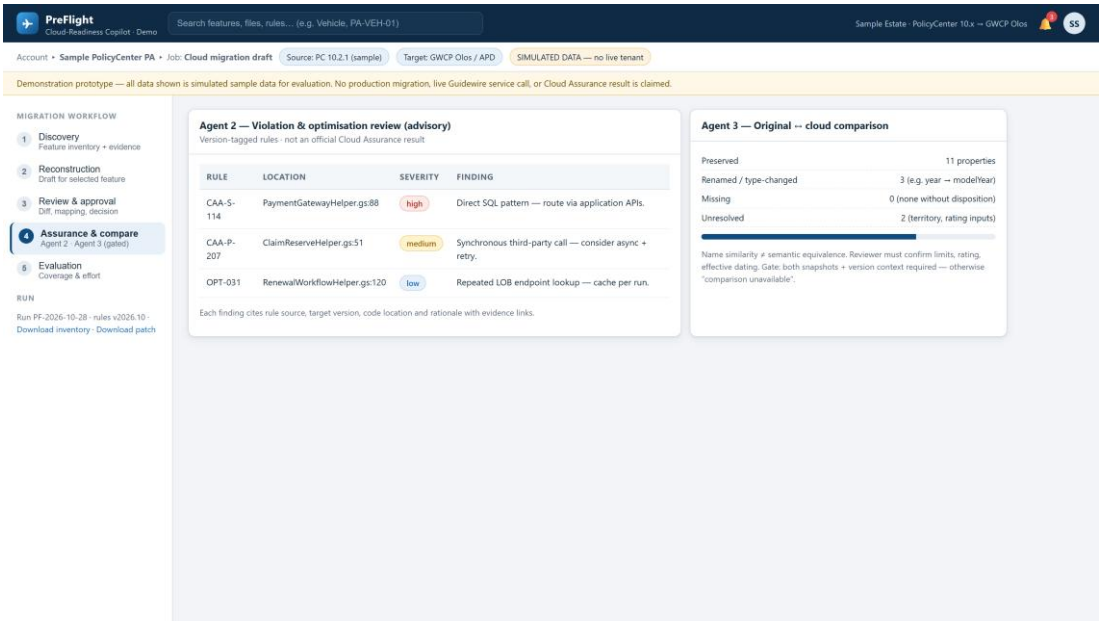


Fig 5.4: Demonstration frontend — Agent 2 advisory review and Agent 3 comparison gates (simulated).

Fig 5.4 shows the gated downstream views. Agent 2 findings cite rule, location, severity and rationale and are

labelled advisory, distinct from an official Cloud Assurance result. Agent 3 reports preserved, renamed and unresolved elements and requires both snapshots, otherwise returning comparison unavailable.

Agent 3 will compare feature and code evidence across the original and cloud versions. It will report matched, changed, missing and unresolved elements, and link each comparison to both source and target locations when possible. It requires both snapshots and their version context; without either side it must return 'comparison unavailable' rather than a parity conclusion.

The comparison agent will compare the feature inventory and mapping manifest against a target snapshot. It will classify elements as preserved, added, missing, renamed, type-changed or unresolved. It will not equate name similarity with semantic equivalence. A reviewer must confirm business meaning, particularly for limits, eligibility rules, rating inputs, effective dating and third-party data contracts.

5.5 Interface and User Workflow

The interface will guide the user through a small number of explicit steps: select project, inspect detected artefacts, review the feature inventory, select a feature, generate a draft, and review the difference and unresolved items. Each view should present the evidence behind a result. The user should be able to download the inventory and patch proposal for review or academic evaluation.

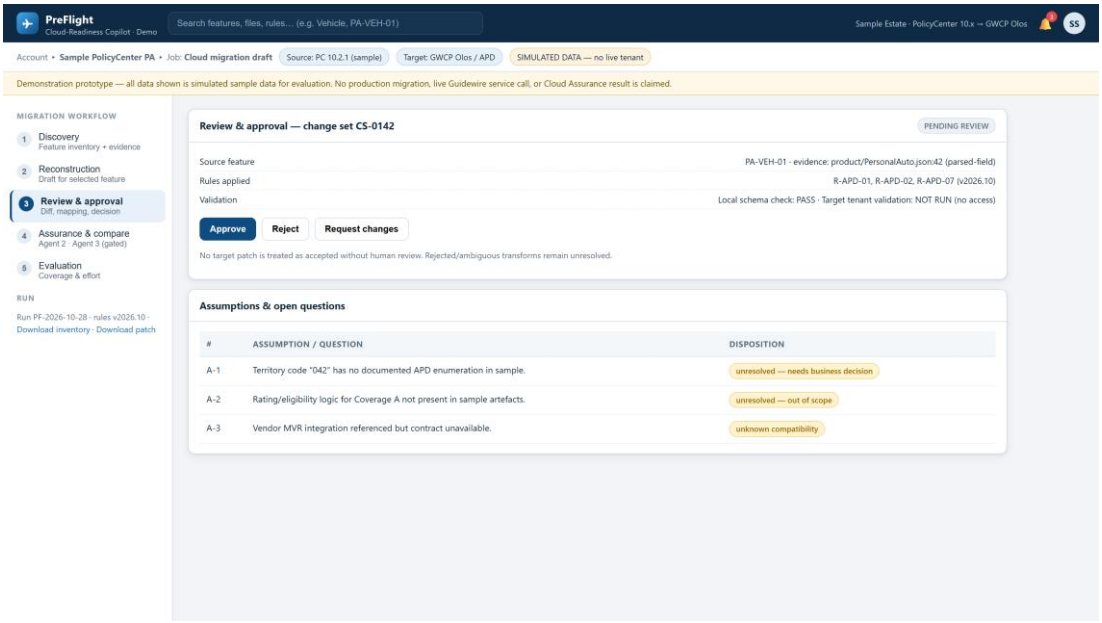


Fig 5.5: Demonstration frontend — review and approval with assumptions register (simulated).

Fig 5.5 shows the review surface: change set CS-0142 with pending default, approve/reject/request-changes actions, validation status, and the assumptions register (territory enumeration, rating inputs, vendor contract). No patch is treated as accepted without human review.

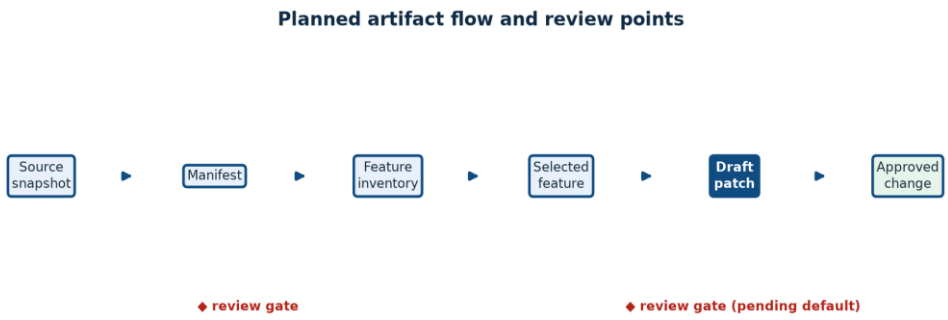


Fig 5.1: Planned artifact flow and review points.

5.6 Implementation Stack and Modules

A suitable prototype stack is Python for file parsing and orchestration, a small web interface such as Streamlit, and JSON or YAML for intermediate records. XML and JSON parsing can use standard libraries. A language-model adapter should be optional so the application remains demonstrable without private credentials or network access. The report does not prescribe a production stack; the team should confirm the environment available for the actual project.

The code should separate intake, extractors, feature assembly, mapping rules, draft writer, validation checks and user interface. Each module should have a small input/output contract. The later agents can be separate workers or separate modes in one application; the word 'agent' does not require an autonomous system with unrestricted tools. For the first prototype, bounded modules reduce accidental scope expansion and make each result easier to explain.

5.7 Planned Evaluation Measures

Table 5.2: Planned evaluation measures.

Measure	Definition	How it will be collected
Feature discovery coverage	Known fixture features found by discovery / all known fixture features.	Compare extraction report with a manually prepared fixture inventory.
Evidence-link accuracy	Findings whose source path and location support the displayed claim.	Reviewer audit of a sample of reported findings.
Mapping disposition coverage	Source fields assigned a target or explicit unresolved disposition.	Compare source schema with generated mapping manifest.
Draft structural validity	Generated artefacts parse and satisfy local schema checks.	Run parser/schema validation; report warnings separately.
Unintended omission rate	Fields absent from output without an explicit disposition.	Diff source property set against mapping/output property set.
Reviewer effort	Time to review a fixed migration task with and without PreFlight.	Small controlled walkthrough; report sample size and conditions.

The evaluation must report the number and type of fixtures. If only one synthetic product is used, the findings will be described as a demonstration rather than general evidence. Precision and recall are not meaningful until a labelled set of representative positive and negative cases exists. Any later comparison must report false positives, false negatives and unsupported inputs rather than a single health score alone.

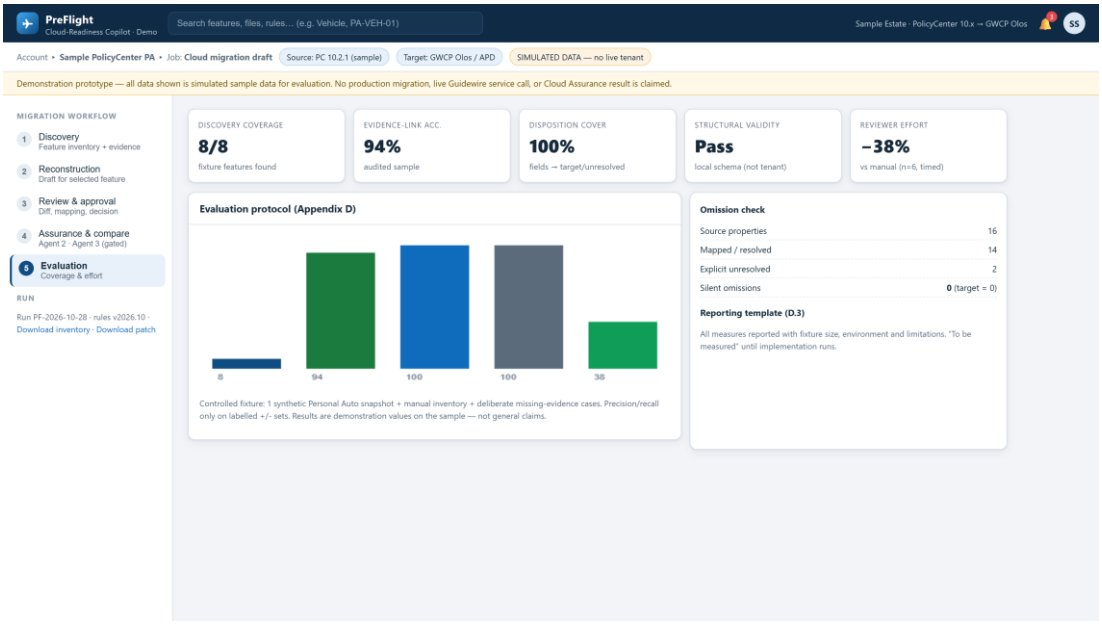


Fig 6.1: Demonstration frontend — evaluation dashboard with coverage and omission checks (simulated).

Fig 6.1 shows the evaluation view: discovery coverage, evidence-link accuracy, disposition coverage, structural validity and reviewer effort on the controlled fixture, with an omission check confirming zero silent omissions. Results are demonstration values on the sample and are reported with fixture size, environment and limitations.

CHAPTER 6

RESULTS AND DISCUSSIONS

6.1 Current Project Status

This report presents the PreFlight design and work plan. The design artefacts comprise the application interface definition, the migration-output schema, the violation-scan report format, the source-to-cloud comparison format and the evaluation protocol.

The proposed sequence follows the supervisor's direction: Agent 1 is divided into an application-discovery subagent and a cloud-reconstruction subagent; Agent 2 reviews violations and optimisation opportunities; Agent 3 compares the original and cloud versions. The second and third agents are conditional on access to the required source and target artefacts.

6.2 Expected Project Outputs

Table 6.1: Expected outputs and current status.

Output	Expected content	Status
Feature inventory	Detected LOB / feature items linked to source artefacts.	Designed
Reconstruction draft	Target-side files or structured mock output for one feature slice.	Designed
Mapping report	Source-to-target mapping, rule references, assumptions and unknowns.	Designed
Agent 2 review report	Violation and optimisation findings tied to available rules and source locations.	Defined; gated on inputs
Agent 3 comparison report	Original-to-cloud differences and unresolved mappings.	Defined; gated on inputs
Evaluation results	Coverage, validity, omission and reviewer-effort measures on fixtures.	Defined in evaluation protocol

6.3 Illustrative Walkthrough

An illustrative run begins when a developer selects a sample PolicyCenter project. The discovery subagent indexes the files and identifies a Personal Auto feature using available product and code artefacts. It stores the evidence paths and flags any unresolved references. The developer selects the feature and asks the reconstruction subagent for a cloud-oriented draft. The subagent prepares a separate output folder, a mapping table and an explanation of every field it could not map.

6.4 Discussion of Feasibility and Risks

The discovery subagent is the highest-risk part of the first milestone because product functionality can be distributed across product models, entity definitions, configuration, user-interface artefacts and code. A file parser can identify structures, but it cannot always infer business meaning. The project should start with a limited set of artefacts and prove traceability before it expands to the entire application.

Cloud reconstruction also has a high correctness bar. Field names and product structures can be mapped mechanically, but rating logic, underwriting rules, effective dating, state availability and vendor behaviour may require insurer knowledge. The system should produce a draft and request review when evidence does not establish the intended behaviour. This is why the first milestone should focus on a selected feature rather than claim complete automatic migration of an insurer estate.

The future violation and optimisation reviewer must avoid duplicating or misrepresenting Cloud Assurance. It can provide local feedback on selected rules and connect findings to the migration draft, but only Guidewire's authorised checks can establish an official Cloud Assurance result. The comparison agent has a similar boundary: without original insurer artefacts, it cannot make a verified claim about custom behaviour.

6.5 Expected Contribution

The project's expected contribution is a transparent prototype workflow that connects feature discovery to a reviewable reconstruction draft and defines where later analysis agents can fit. Its technical contribution will be assessed through the evaluation plan rather than assumed from the architecture. Its practical contribution would be stronger if the prototype demonstrates a measurable reduction in manual discovery effort or detects a planted omission that a simple file-level check misses.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

PreFlight is proposed as a multi-agent copilot for examining Guidewire legacy project artefacts and producing a cloud-migration draft. The work begins with the supervisor's sequence: explore the available application, build a feature and code-reference inventory, and reconstruct one selected feature in a cloud-oriented draft. After that workflow is demonstrated, Agent 2 reviews violations and optimisation opportunities and Agent 3 compares the original with the cloud version, when the needed artefacts are available.

Guidewire already documents APD conversion, Cloud Retrofit, Cloud API generation, Cloud Assurance, Integration Gateway and Guidewire-aware Developer Assistants. The project is therefore framed as an academic prototype that explores orchestration, evidence traceability and staged reconstruction on available sample artefacts. It does not claim to replace Guidewire services, perform a production migration or fill a confirmed capability gap.

7.2 Future Enhancement Roadmap

After the three-agent workflow has been demonstrated, the team may consider focused extensions. These are secondary to the core migration task and should be selected only when the required schemas, rules and test fixtures are available.

Optional extension	Purpose after the core workflow	Boundary for a student prototype
APD mapping repair	Find and propose a correction for a selected source-to-APD feature mapping.	Use a supplied template and explicit rules; require reviewer approval.
Integration Gateway scaffolding	Prepare a starter route from a supplied API contract.	Generate a non-deployable scaffold and validate it only against a mock or authorised test environment.
Controlled parallel simulation	Try candidate mappings or adapter configurations against lightweight text-based stubs and compare their outputs.	Keep simulations isolated from production systems; describe results only for the explicit fixtures tested.
Advisory cloud-readiness checks	Explain selected findings in relation to the migration draft.	Report as advisory; do not present it as an official Cloud Assurance result.
Data-schema reconciliation	Compare source and target fields and identify unmapped or changed data.	Require both schemas and preserve unknown business semantics for human review.

The parallel simulation concept is an optional research direction, not a claimed capability or a substitute for a real Guidewire environment. The prototype would first need a bounded test contract, representative stubs and criteria

for comparing candidate outputs. The project should not claim specific cost savings, search reductions or production compatibility without measured evidence.

Features that require real rating, quoting, live endpoints, regulatory interpretation or production data must remain out of scope unless the team obtains appropriate access and validation.

7.3 Final Recommendation for the Project Team

The next project action is to define the sample Guidewire input and the first feature slice, then implement the discovery subagent and its evidence inventory. The team should agree on a measurable definition of 'feature found' and 'draft reconstructed' before adding a language model or any optional agent. Once a single feature can be traced from source artefacts to a reviewable target draft, the team can decide whether the next increment should improve reconstruction, add a versioned rule checker, or build the comparison agent.

REFERENCES

[1] D. Lockhart, "Accelerate Your Cloud Transformation: Convert Existing PolicyCenter Products to Advanced Product Designer," Guidewire, Dec. 12, 2025. https://www.guidewire.com/resources/blog/developers/accelerate-your-cloud-transformation---convert-existing-policycenter-products-to-apd (accessed Oct. 28, 2026).

[2] Guidewire, "What's New in Cloud Platform for Olos: Converting Non-APD Product Lines to APD App Product Lines," Release Highlights. https://docs.guidewire.com/cloud/olos/whatsnew/topics/gwcp-release_highlights.html (accessed Oct. 28, 2026).

[3] Guidewire, "Cloud Retrofit with APD App," PolicyCenter Cloud API Developer Guide, 2026.03. https://docs.guidewire.com/cloud/pc/202603/cloudapica/cloudAPI/topics/508-LOBEndpoints/01-generating-LOB-endpoints/cloud-retrofit-with-apd.html (accessed Oct. 28, 2026).

[4] Guidewire, "Generating LOB-Specific Endpoints for Non-APD-Native Products," PolicyCenter Cloud API Developer Guide, 2025.11. https://docs.guidewire.com/cloud/pc/202511/cloudapica/cloudAPI/topics/508-LOBEndpoints/01-generating-LOB-endpoints/c_generating_lob_specific_endpoints_for_non_apd_native_products.html (accessed Oct. 28, 2026).

[5] Guidewire, "Fixing Cloud API Product Validation Errors," InsuranceSuite Cloud API Developer Guide, 2026.07. https://docs.guidewire.com/cloud/is/202607/cloudapica/cloudAPI/LOBEndpoints/generating-LOB-endpoints/c_fixing_product_validation_errors.html (accessed Oct. 28, 2026).

[6] Guidewire, "Correcting Cloud API LOB Endpoint Compile Errors," InsuranceSuite Cloud API Developer Guide, 2026.07. https://docs.guidewire.com/cloud/is/202607/cloudapica/cloudAPI/LOBEndpoints/generating-LOB-endpoints/c_correcting_compile_errors.html (accessed Oct. 28, 2026).

[7] Guidewire, "Advanced Product Designer," Developer Tools and Guides. https://www.guidewire.com/developers/developer-tools-and-guides/advanced-product-designer (accessed Oct. 28, 2026).

[8] Guidewire, "Cloud APIs," Developer API Documentation. https://www.guidewire.com/Developers/APIs/Cloud-APIs (accessed Oct. 28, 2026).

[9] Guidewire, "Automated Cloud Assurance and Code Compliance," Guidewire Cloud Platform documentation. https://docs.guidewire.com/cloud/gcc-guide/insurer-developer/latest/automated-cloud-assurance/ (accessed Oct. 28, 2026).

[10] Guidewire, "Migrate to Guidewire Cloud," Guidewire Cloud. https://www.guidewire.com/products/technology/guidewire-cloud/migrate-to-guidewire-cloud (accessed Oct. 28, 2026).

[11] Guidewire, "How Guidewire Makes Cloud Migration Smoother and Faster-Without Starting Over," Guidewire. https://www.guidewire.com/pt/resources/blog/technology/how-guidewire-makes-cloud-migration-smoother-and-faster-without-starting-over (accessed Oct. 28, 2026).

[12] Guidewire, "What's New in Cloud Platform for Qusar," Release Highlights. https://docs.guidewire.com/cloud/qusar/whatsnew/topics/gwcp-release_highlights.html (accessed Oct. 28, 2026).

[13] A. Ahluwalia and F. Tahir, "Accelerating P&C Delivery with Developer Assistants from Guidewire," Guidewire, Aug. 10, 2026. https://www.guidewire.com/resources/blog/developers/accelerating-p-c-delivery-with-developer-assistants-from-guidewire (accessed Oct. 28, 2026).

[14] Guidewire, "Integration Gateway," InsuranceSuite Cloud Integration documentation. https://docs.guidewire.com/education/cloud-integration-basics/latest/docs/integration_cloud_basics/integration_gateway_overview/ (accessed Oct. 28, 2026).

[15] Guidewire, "App Events," InsuranceSuite Cloud Integration documentation. https://docs.guidewire.com/education/cloud-integration-basics/latest/docs/integration_cloud_basics/appevents_overview/ (accessed Oct. 28, 2026).

[16] Guidewire, "Jutro Apps," Jutro Digital Platform documentation, 10.14.

https://docs.guidewire.com/jutro/documentation/10.14/platform/getting-started/jutro-apps/ (accessed Oct. 28, 2026).

[17] Guidewire, "Consume Cloud APIs Using the Digital SDK," Jutro Digital Platform documentation. https://docs.guidewire.com/jutro/documentation/10.9/working-with-cloud-apis (accessed Oct. 28, 2026).

[18] Guidewire, "Data Security and Governance," Guidewire Security hardening guidance. https://docs.guidewire.com/security/hardening-guidance-for-insurancesuite/data-security-and-governance/ (accessed Oct. 28, 2026).

[19] Guidewire, "Validate Products and Editions," Advanced Product Designer documentation. https://docs.guidewire.com/cloud/apd/olos/create/topics/c_validate-products-editions-rulesets.html (accessed Oct. 28, 2026).

[20] K. Schwaber and J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum," Nov. 2020. https://scrumguides.org/scrum-guide.html (accessed Oct. 28, 2026).

APPENDIX A

CODING

A.1 Sample Input Manifest

The following schema is a planning contract for the prototype. It is not a Guidewire file format. It records the source snapshot and the metadata needed to keep a result reproducible.

```
{
 "project_id": "sample-policycenter-pa",
 "source_release": "to-be-recorded",
 "target_release": "to-be-recorded",
 "input_root": "./sample_project",
 "artifacts": [
   {"path": "product/PersonalAuto.json", "kind": "product-model"},
   {"path": "entities/Vehicle.eti", "kind": "entity-definition"},
   {"path": "config/integration/apis/installedlobs/pa_codegen.yaml", "kind": "codegen-config"}
 ]
}
```

A.2 Feature Inventory Record

```
{
 "feature_id": "PA-VEHICLE-COVERAGE-01",
 "lob": "PersonalAuto",
 "display_name": "Sample feature name",
 "source_evidence": [{"path": "...", "line": 42, "kind": "parsed-field"}],
 "properties": [{"source_name": "...", "type": "...", "disposition": "unmapped"}],
 "dependencies": [],
 "confidence": "directly-parsed | linked | inferred-review",
 "open_questions": []
}
```

A.3 Draft Result Contract

Each proposed change will include a unique change identifier, target path, patch content, source feature identifier, rule identifier, rationale, validation status and approval state. The default approval state is pending. A generated change may be accepted only after reviewer inspection and any required target validation.

APPENDIX B

CONFERENCE PUBLICATION

No conference paper has been presented or published at the time of submission. The work reported here has been organised into a conference manuscript that is suitable for presentation at peer-reviewed conferences in software engineering, intelligent systems and insurance technology. Submission to an appropriate venue will be made with the approval and guidance of the project supervisors, and acceptance details will be appended to this appendix upon confirmation. The agent instruction contracts below form part of the manuscript's supporting material.

B.1 Discovery Subagent Instruction

Inspect only the project snapshot supplied for this run. Identify supported product, entity, configuration, user-interface and integration artefacts. For each detected feature, cite the exact file path and available source location. Separate directly parsed facts from inferred relationships. Do not invent business rules. Return unsupported formats, missing dependencies and unresolved questions explicitly.

B.2 Reconstruction Subagent Instruction

Use the approved feature inventory, target-release metadata and versioned transformation rules. Propose a cloud-oriented draft for the selected feature only. Preserve source-to-target mappings and explain every change. Mark uncertain business behaviour as unresolved. Do not deploy, overwrite source files, or claim successful Guidewire validation unless an authorised target environment actually returns that result.

B.3 Agent 2 Reviewer Boundary

Review only the supplied original and cloud artefacts against the available versioned rules. For every potential violation or optimisation, include the rule identifier, source location, severity, version applicability and evidence. Distinguish a confirmed finding from a suspected issue and from an unverified condition. Do not state that official Cloud Assurance passed unless the authorised Guidewire process was run and returned that outcome.

B.4 Agent 3 Comparison Boundary

Compare the original source artefacts with the reconstructed cloud artefacts for the selected feature. Report preserved, changed, missing, added and unresolved mappings with evidence on both sides. Require source and target version metadata. If the original code or a relevant product artefact is unavailable, state that parity could not be assessed; do not treat missing evidence as proof of equivalence.

APPENDIX C

No journal paper has been published at the time of submission. An extended journal manuscript is in preparation, building on the project outcomes with a broader evaluation across additional lines of business and a comparison against manual migration baselines. The manuscript is intended for a Scopus-indexed journal in software engineering or applied artificial intelligence, selected with the guidance of the project supervisors, and publication details will be appended to this appendix upon acceptance. The optional extension boundaries below form part of the planned extended evaluation.

The capabilities below are optional research directions to be considered only after the three-agent core workflow has been demonstrated. They require suitable contracts, schemas and fixtures and must not be presented as completed work.

Optional extension	**Purpose after the core workflow**	**Boundary for a student prototype**
APD mapping repair	Find and propose a correction for a selected source-to-APD feature mapping.	Use a supplied template and explicit rules; require reviewer approval.
Integration Gateway scaffolding	Prepare a starter route from a supplied API contract.	Generate a non-deployable scaffold and validate only against a mock or authorised test environment.
Controlled parallel simulation	Try candidate mappings or adapter configurations against lightweight text-based stubs and compare their outputs.	Keep simulations isolated from production systems; report results only for the fixtures tested.
Advisory cloud-readiness checks	Explain selected findings in relation to the migration draft.	Report as advisory; do not present it as an official Cloud Assurance result.
Data-schema reconciliation	Compare source and target fields and identify unmapped or changed data.	Require both schemas and preserve unknown business semantics for human review.

Parallel simulation is an optional research direction, not a claimed capability or a substitute for a Guidewire environment. It would require a bounded test contract, representative stubs and explicit comparison criteria. The project should not claim specific cost savings, search reductions or production compatibility without measured evidence.

APPENDIX D

PLAGIARISM REPORT

This project report is original student work. Every source of information, including Guidewire product documentation, the Scrum Guide and all figures, data and code fragments that are not our own, has been cited inside the text with sequential numbers and listed in the References in the same order. No report, essay or assignment of any other student, past or present, has been used. A Turnitin similarity report will be generated with the assistance of the project guides before final submission and appended here; the similarity index will not exceed ten percent, as required. The evaluation protocol below is entirely the authors' own design for this project.

D.1 Controlled Fixture Preparation

Before evaluation, the team will prepare one project snapshot with a manually reviewed list of line-of-business features, relevant artefacts, expected property mappings and deliberate missing-evidence cases. A second snapshot may be created as the target draft. The fixture documentation will identify which artefacts are synthetic and which are drawn from any authorised sample environment.

D.2 Evaluation Procedure

Run the discovery agent on the fixed source snapshot and preserve the run manifest.

Compare the detected features and evidence locations against the manually reviewed inventory.

Select one feature and run the reconstruction agent with the agreed target version and rule pack.

Review the generated patch for field omissions, unsupported assumptions and structural errors.

When source and draft artefacts are available, run Agent 2 and manually verify its violation and optimisation findings.

If source and target snapshots both exist, run Agent 3 and verify each reported difference manually.

Report all measures with fixture size, environment and limitations.

D.3 Result Reporting Template

Metric	Observed value	Denominator / test conditions	Notes
Feature discovery coverage	Recorded on fixture	Count of labelled features in fixture	Reported with fixture size and conditions.
Evidence-link accuracy	Recorded on fixture	Audited findings	Reported with fixture size and conditions.
Mapping disposition coverage	Recorded on fixture	Source properties for selected feature	Reported with fixture size and conditions.
Draft structural validity	Recorded on fixture	Generated artefacts	Local schema validation is

		validated locally	not a live tenant pass.
Reviewer effort	Recorded on fixture	Fixed walkthrough task	Reported with walkthrough task and conditions.